

25/09 - Testing

Testing

Es un proceso destructivo, cuyo propósito es encontrar defectos que se presume que se encuentran en el código.

Y testing exitoso es aquel que encuentran un defecto

Se testea la versión de un producto de software (código) que tiene un conjunto de funcionalidades.

Comparación de versiones

Comparación de la versión del código, contra los requerimientos. Lo que utilizamos para realizar el testeo son los **casos de pruebas**

- Los casos de pruebas son los artefactos que surgen del testing. Eso genera el testing sistemático, el testing no sistemático es aquel que surge de hacer pruebas sin guía o sin dejar por sentado lo que se hace (escenarios).
- Un error que no se puede replicar, no es considerado un error (surge por lo general en el testing no sistemático, ya que no hay un paso a paso de cómo lograr ese resultado)

Administración de software

- **QA ≠ Testing** (el aseguramiento de calidad, no solo prueba calidad del producto, sino también del proceso. Además asegura el control del mismo.
- El testing no asegura ni garantiza calidad (solo busca defectos en el producto de software, y es muy probable que no se encuentren todos)

Error vs defectos

La diferencia está en cuando aparece. Si se encuentran en el momento en el que se introdujo, es un error. Si se pasó de etapa del PUD y no se encontró el problema en la etapa que lo introdujo es un defecto (significa que se trasladó el problema)

Siempre se busca una detección temprana porque es mucho más barato corregir algo en ese momento, a que después de haber hecho todo (porque ahí ya se perdió plata y tiempo)

→ **Falla**, es aquello que produce un defecto en el software. Pero no todos los defectos producen una falla, ya que tranquilamente el sistema sigue

funcionando.

Severidad del defecto (qué tan **grave** es para la utilidad del sistema) (si funciona o no)

- Bloqueante (no permite utilizar el sistema)
- Crítico (requiere atención urgente)
- Mayor (error que impide cumplir con el objetivo de una funcionalidad, pero se puede hacer de otra forma)
- Menor (error que lanza una alerta pero se ejecuta igualmente)
- Cosmético (mal uso de colores, errores de tipeo, errores de ortografía, no ingreso de datos opcionales)

Prioridad defecto (cuan **necesario** es para el usuario final (cliente) que el sistema funcione)

- Urgencia (hay que solucionarlo lo antes posible, ya mismo)
- Alta
- Media
- Baja

Niveles de prueba

Determinan el **orden** en el que se van a probar las cosas

1. Pruebas unitarias, son las que hace el desarrollador (excepción a la regla) en lugar del tester. Durante el momento de desarrollo
2. Pruebas de integración, se realizan durante el desarrollo o en la fase de prueba (prueba)
3. Pruebas de sistemas, se realizan en la etapa de testing por los testers
4. Pruebas de aceptación del usuario de reslizan en la etapa de despliegue.

TDD, test driven development. Pruebas conducidas en el desarrollo

Testing unitario

Se prueban componente por componente

Se prueban componentes individuales y de manera independiente

Los errores de suelen reparar tan pronto se encuentran, sin constancia oficial de los incidentes.

Testing de integración

Se prueban la interfaz, busca que los componentes que se probaron individualmente, funcionan ahora que están conectados en un build (integración). Lo lleva a cabo un arquitecto o project builder.

Testing de sistema

Pruebas de version, (todo lo que se construyó en la primera iteración sería la primera versión), se espera que lo realice alguien que no haya participado en el desarrollo (persona distinta del equipo, persona de otro equipo, gente independiente o incluso externa)

Testing de aceptación

Las lleva a cabo el usuario del sistema. Si se llega a encontrar algún defecto en el medio, el usuario debe reportarlo. Es el último paso antes del cobro.

Ambientes (entornos) para construcción de software

Es el repositorio configurado, involucra a todo lo que es necesario para poder construir.

Mínimamente deben existir 3 ambientes, porque el de pre producción es un calco de producción para hacer pruebas de usuario

1. Ambiente de desarrollo; solo lo prueban las personas que desarrollan.

Pruebas unitarias y pruebas de integración

2. Pruebas; solo tienen permisos las personas que van a probar (debe tener todo lo mismo que tenía el desarrollo (obviamente sin poder modificar nada)). De esa forma se elimina el concepto de "en mi máquina andaba".

Pruebas de sistemas

3. Pre producción (permite hacer pruebas de usuario antes de desplegarlo oficialmente). **Pruebas de aceptación**
4. Producción. **Pruebas de aceptación**

Caso de prueba

Set de condiciones o variables bajo las cuales un tester determinará si el software está funcionando correctamente o no

Una buena definición de caso de pruebas nos ayuda a reproducir defectos.

Los bugs, se esconden en las esquinas (los problemas están en los bordes o límites)

Condiciones de prueba

Estrategias de pruebas

- Testing de caja negra. Se ejecuta y el sistema arroja un resultado.
 - Posee métodos
- Testing caja blancos. Es estática, mirando el código.
 - Posee métodos de testeo de: cobertura de enunciados, cobertura de sentencias, cobertura de decisión, cobertura de condición, cobertura múltiple etc.

Cada técnica tiene un nivel de abstracción diferente, y resultan ser complementarias entre sí. La caja negra nos permite ver si lo que sale es realmente lo que esperábamos, pero con la caja blanca vemos si hay código malicioso, si hay algo erróneo.

Ciclo de test

Un ciclo de prueba abarca la ejecución de la totalidad de los casos de prueba establecidos aplicados a una misma versión del sistema a probar

Regresión

- Sin regresión
- Con regresión

Proceso de pruebas

Planificación, se planifica la prueba (requerimientos del sistema)

Etapa de preparación (diseño), donde defino casos de pruebas. Se busca fuente de información, con datos reales. Involucra identificación y especificación

Llegando a la versión, hacemos la etapa de ejecución para buscar defectos

Una vez que encontramos estos defectos, realizamos el análisis de fallas

Y se llega a un momento donde le damos fin a las pruebas, asegurando la mayor parte del sistema, y se toma según temas de costo y tiempo.